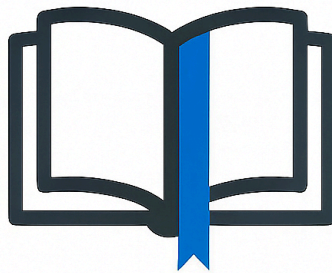


BookMarker
Test Plan
Versione 0.2



Bookmarker

Data: 20/12/2025

Progetto: BookMarker	Versione: 0.1
Documento: Test Plan	Data: 20/12/2025

Coordinatore del progetto:

Nome	Matricola

Partecipanti:

Nome	Matricola
Francesco Liguori	0512109651
Antonio Prisco	0512123502
Nicholas Franco Grimaldi	0512109702

Scritto da:	Tutti i partecipanti
--------------------	----------------------

Revision History

Data	Versione	Descrizione	Autore
20/12/2025	0.1	Prima stesura	Tutti i partecipanti
21/12/2025	0.2	Aggiunta paragrafo 6,7,8	Antonio Prisco

Indice

1. Introduzione	4
2. Relazione con gli altri documenti	4
3. Panoramica del sistema	4
4. Funzionalità da testare/da non testare	5
5. Criteri di fallimento	6
6. Approccio	6
7. Regole di sospensione e ripresa del testing	7
8. Strumenti per il testing	8
9. Test Case	8

1. Introduzione

L'obiettivo di questo documento è quello di definire le strategie che verranno utilizzate per effettuare il testing del sistema BookMarker, andremo ad analizzare i componenti specificando la strategia di testing che utilizzeremo e i relativi casi di test.

2. Relazione con gli altri documenti

Il Test Plan è in relazione con i documenti prodotti finora, in particolare si farà riferimento ai requisiti funzionali del RAD, e i test case verranno presi dai casi d'uso.

Tramite il System Design Document e l'Object Design Document definiremo poi i sottosistemi e i servizi, le classi e le componenti del sistema da testare.

3. Panoramica del sistema

Il sistema BookMarker ha lo scopo di fornire un sito web che permette di facilitare l'interazione tra una persona e la biblioteca. È possibile visualizzare in tempo reale i libri attualmente disponibili, controllare lo storico dei prestiti effettuati, verificare la data di restituzione dei libri attualmente in possesso, e lasciare recensioni sui libri letti.

4. Funzionalità da testare/da non testare

Funzionalità da testare:

Visitatore

- RF_1.1 - Registrarsi al sito
- RF_1.3 - Cercare un libro attraverso ricerca o filtri

Lettore

- RF_2.1 - Effettuare l'autenticazione al sito
- RF_2.3 - Cercare un libro attraverso ricerca o filtri
- RF_2.6 - Lasciare recensioni ai libri presi in prestito
- RF_2.7 - Segnalare recensioni non appropriate

Moderatore

- RF_3.5 - Gestire le segnalazioni degli utenti
- RF_3.6 - Rimuovere recensioni non appropriate

Gestore

- RF_4.4 - Modificare la disponibilità di un libro
- RF_4.5 - Assegnare il prestito di un libro a un utente
- RF_4.7 - Aggiungere e rimuovere un libro dal catalogo

Funzionalità da non testare:

Visitatore

- RF_1.2 - Visualizzare il catalogo dei libri disponibili
- RF_1.4 - Consultare le recensioni dei libri

Lettore

- RF_2.2 - Visualizzare il catalogo dei libri disponibili
- RF_2.4 - Consultare le recensioni dei vari libri
- RF_2.5 - Consultare il proprio storico dei libri presi in prestito

Moderatore

- RF_3.2 - Visualizzare la lista delle segnalazioni degli utenti
- RF_3.4 - Visualizzare la lista delle recensioni

Gestore

- RF_4.2 - Visualizzare il catalogo gestore dei libri
- RF_4.6 - Visualizzare la pagina dello storico globale dei prestiti

5. Criteri di fallimento

Un Test Case fallisce se e solo se l'output osservato durante l'esecuzione coincide perfettamente con l'output atteso definito nella specifica del test. Un Test Case ha successo se si verifica una qualsiasi discrepanza tra il risultato ottenuto e quello atteso, inclusi malfunzionamenti come messaggi di errore imprevisti o dati non corretti nel database.

6. Approccio

I livelli di test previsti sono:

- **Testing di Unità:** per testing di unità si intende il testing delle singole unità software del sistema, il test verrà effettuato con il framework JUnit. Per Individuare i test da effettuare si utilizzerà la tecnica Black-Box.
- **Testing di Integrazione:** l'approccio che verrà utilizzato è il bottom-up, ritenuto il più adatto per i software basati sul paradigma Object Oriented. Si farà utilizzo del framework JUnit per la definizione dei test case in Java.
- **Testing di Sistema:** questa fase ha l'obiettivo di verificare il comportamento del software nel suo complesso rispetto ai requisiti funzionali e non funzionali definiti nel RAD. Il testing di sistema verrà realizzato tramite il tool Selenium, che permette di registrare le azioni che un utente svolge sul browser per implementare ed eseguire i test case.

La loro esecuzione avverrà durante la stesura del codice, in modo parallelo, così da avere un riscontro immediato degli errori di implementazione. Ogni volta che verrà introdotta una nuova funzionalità verranno eseguiti i test ad essa legata e, inoltre, i test di integrazione che la coinvolgono. Ad ogni modifica delle componenti, verranno eseguiti i test di unità e di integrazione delle componenti direttamente dipendenti da quelle modificate.

7. Regole di sospensione e ripresa del testing

Regole di sospensione

La fase di testing verrà sospesa nei seguenti casi:

- **Rilevamento di difetti bloccanti:** qualora venga individuato un errore critico (fault) che impedisca il normale proseguimento delle attività di test o limiti l'accesso a funzionalità fondamentali (ad esempio, un errore nel sottosistema di autenticazione che impedisce l'accesso a tutte le altre aree).
- **Rischi per le risorse:** la sospensione deve essere attuata se il proseguimento dei test comporta un impatto eccessivo o uno spreco delle risorse disponibili senza produrre risultati utili.
- **Raggiungimento degli obiettivi:** il processo può essere sospeso se tutti gli obiettivi dichiarati nel Test Plan sono stati raggiunti entro i tempi prefissati.

Regole di Ripresa

L'attività di testing riprenderà non appena le condizioni che hanno causato la sospensione saranno rimosse:

- **Risoluzione del difetto:** la ripresa avviene solo dopo che il difetto segnalato è stato corretto con successo.
- **Verifica della stabilità:** una volta ripreso il processo, i test precedentemente falliti verranno ripetuti.
- **Testing di regressione:** verranno rieseguiti anche i test sulle componenti correlate per garantire che le modifiche e le correzioni non abbiano introdotto nuovi errori (regressioni) in parti del sistema precedentemente stabili.

8. Strumenti per il testing

Gli strumenti necessari per svolgere le attività di testing sono:

- **WebServer Apache Tomcat 9** in locale, necessario per il deploy del sistema e l'esecuzione dell'applicazione in un ambiente controllato durante le fasi di test.
- **Un DBMS MySQL**, che si occupa della gestione della persistenza dei dati e dell'interazione con il database.
- **Selenium IDE**, utilizzato come tool per il testing di sistema e di integrazione, in quanto permette di registrare e riprodurre le azioni che un utente svolge sul browser.
- **JUnit**, il framework di riferimento per lo svolgimento dei test di unità sui singoli metodi delle classi e per la verifica dell'integrazione tra i sottosistemi.

9. Test Case